

Embedded Linux & System Software Interview Handbook

Chapter 7 - Linux Kernel Synchronization & Concurrency (Part 6)

Topics: Per-CPU Data, Lock-Free Queues, Performance Optimization & Case Studies

7.12 Per-CPU Data

Per-CPU variables provide each CPU with its own copy of data, eliminating contention and reducing cache-line bouncing. They are heavily used for statistics, networking and scheduler data.

Common APIs

API	Purpose
DEFINE_PER_CPU()	Declare per-CPU variable
this_cpu_read()	Read local CPU value
this_cpu_write()	Write local CPU value
this_cpu_inc()	Increment local counter
for_each_possible_cpu()	Iterate CPUs

Lock-Free Queues

High-performance kernel subsystems often use lock-free ring buffers built with atomic operations and memory barriers to minimize latency and improve scalability.

Performance Guidelines

- Prefer finer-grained locking.
- Minimize lock hold time.
- Avoid false sharing.
- Use per-CPU data where appropriate.
- Profile before optimizing.

Real Driver Case Studies

Network drivers protect TX/RX rings with NAPI and per-CPU structures. Block drivers synchronize request queues carefully. UART drivers combine spinlocks, wait queues and interrupt handlers for efficient serial communication.

Debugging Concurrency Issues

Tool	Usage
perf	Contention and performance analysis
ftrace	Kernel execution tracing
lockstat	Lock contention statistics
KCSAN	Kernel Concurrency Sanitizer

KASAN	Memory error detection
syzkaller	Concurrency bug discovery

Senior Interview Questions

1. What are per-CPU variables and why are they faster?
2. How do you reduce lock contention?
3. Explain false sharing.
4. When would you choose lock-free programming over locks?
5. How would you debug an intermittent SMP race?
6. Describe synchronization in a network or UART driver.

Chapter Summary

This chapter covered the complete Linux kernel synchronization ecosystem: spinlocks, mutexes, semaphores, completions, atomic operations, memory barriers, wait queues, RCU, per-CPU data, deadlock debugging and performance tuning. These topics are essential for senior Embedded Linux, BSP and Kernel Developer interviews.